

Documentacion Tecnica del Proyecto

Defensa y Asalto de Base

Estudiante: Nazareth Ulate Diaz

Curso: Introduccion a la Programacion - Modalidad Live Learning

Carrera: Ingenieria en Computacion

1. Atributo de Analisis de Problema

Descripcion del Problema Complejo de Ingenieria

El problema abordado consiste en el diseno e implementacion de un sistema de software interactivo que simule una dinamica de estrategia en tiempo real entre dos jugadores. Desde el punto de vista de la ingenieria de software, el reto principal radica en modelar correctamente entidades del mundo real (torres, unidades, base, jugadores) mediante principios de programacion orientada a objetos, asegurando que sus interacciones sean coherentes, escalables y eficientes. Matematicamente, el problema involucra el calculo de distancias en una matriz bidimensional para determinar el alcance de las torres, la gestion de variables numericas (vida, dano, dinero) y la logica de condiciones de victoria. En cuanto al desarrollo sostenible, el proyecto utiliza herramientas de codigo abierto (Python, Tkinter) que no requieren licencias comerciales, reduciendo costos y barreras de acceso al software.

Analisis del Contexto, Variables y Factores

El proyecto se desarrolla en el contexto de un curso universitario de introduccion a la programacion. Las variables principales del sistema son:

- Estado del mapa: matriz 10x10 que almacena la posicion de cada elemento.
- Economia del juego: dinero de cada jugador, costos de torres y unidades.
- Estadisticas de combate: vida, dano y alcance de cada entidad.
- Progresion de partida: rondas ganadas y condiciones de victoria.
- Persistencia de datos: victorias almacenadas en archivos JSON.

Desde el enfoque de sostenibilidad social, el juego fomenta el pensamiento estrategico y la interaccion presencial entre dos personas, sin necesidad de conexion a internet ni dispositivos de alto costo.

Plan de Solucion Propuesto

La solucion se estructuro bajo el paradigma de programacion orientada a objetos, dividiendo el sistema en clases independientes: Jugador, Torre y sus subtipos, Unidad y sus subtipos, BaseCentral, Muro y un modulo de combate. La interfaz grafica se implemento con Tkinter usando un Canvas como representacion del mapa. El flujo sigue una arquitectura por ventanas (login, seleccion de faccion, seleccion de rol, juego, ranking), donde cada modulo es independiente y se comunica mediante callbacks. Los datos de jugadores se persisten en un archivo JSON.

Evaluacion de Ventajas y Desventajas

Aspecto	Ventajas	Desventajas
Tecnico	POO facilita la extension del codigo. Tkinter incluido en Python.	Tkinter tiene limitaciones graficas comparado con motores de juego.
Economico	Herramientas gratuitas y de codigo abierto.	Requiere Python instalado para ejecutarse.
Ambiental	No requiere servidores ni conexion a internet.	Limitado a Windows por uso de winsound.

Social	Fomenta interaccion presencial entre jugadores.	Solo disponible para 2 jugadores locales.
--------	---	---

2. Atributo de Herramientas de Ingenieria

Tecnicas, Recursos y Herramientas Utilizadas

- **Python 3.10+:** Lenguaje principal. Permite POO, manejo de archivos y uso de bibliotecas estandar.
- **Tkinter:** Biblioteca grafica incluida en Python. Usada para crear ventanas, canvas del mapa y controles.
- **JSON:** Formato de almacenamiento de datos de jugadores con el modulo json de Python.
- **winsound:** Modulo estandar de Python para Windows, usado para reproducir musica de fondo en formato WAV.
- **Git / GitHub:** Control de versiones para registrar el progreso del proyecto con commits descriptivos.
- **POO con Herencia:** Las clases TorreBasica, TorrePesada y TorreMagica heredan de Torre. Igual para las unidades.
- **Patron Callback:** Las ventanas se comunican mediante funciones callback, manteniendo bajo acoplamiento.
- **Matriz 2D:** El mapa se representa como una lista de listas 10x10 con referencias a los objetos del juego.

Diagrama de Clases

El siguiente diagrama representa las principales relaciones entre clases del proyecto:

Clase	Hereda de	Atributos principales	Metodos principales
Jugador	-	usuario, contrasena, victorias_def, victorias_atk, dinero, rol, faccion	dar_dinero(), gastar_dinero(), ganar_ronda(), ganar_partida()
Torre	-	nombre, costo, vida, danio, alcance, faccion, fila, columna	atacar(), puede_atacar(), habilidad_especial()
TorreBasica	Torre	turnos_habilidad = 3	habilidad_especial() - disparo doble
TorrePesada	Torre	turnos_habilidad = 4	habilidad_especial() - dano en area
TorreMagica	Torre	turnos_habilidad = 3	habilidad_especial() - congela unidad
Unidad	-	nombre, costo, vida, danio, velocidad, faccion, congelada	mover(), atacar_torre(), atacar_base(), habilidad_especial()
Soldado	Unidad	turnos_habilidad = 3	habilidad_especial() - ataque doble
Tanque	Unidad	escudo_activo	habilidad_especial() - escudo temporal
UnidadRapida	Unidad	boost_activo	habilidad_especial() - aumento velocidad
BaseCentral	-	vida, vida_maxima, faccion, fila, columna	recibir_danio(), esta_destruida(), reparar()
Muro	-	vida, vida_maxima, faccion, fila, columna	recibir_danio(), esta_destruido()

Aplicacion de las Herramientas

Tkinter se uso para construir una interfaz modular dividida en cinco ventanas independientes. El Canvas fue fundamental para dibujar el mapa dinamicamente en cada turno de combate. La POO permitio que el sistema de combate fuera generico: el modulo combate.py opera sobre cualquier lista de torres y unidades sin importar su tipo, aprovechando el polimorfismo mediante el metodo habilidad_especial() definido en cada subclase.

Adaptacion y Refinamiento de Herramientas

Durante el desarrollo se realizaron varios ajustes. Inicialmente se planifico usar pygame para la musica, pero al presentar incompatibilidades se migro a winsound con threading para no bloquear la interfaz grafica. El sistema de combate se refino para incluir numeros de dano flotantes y barras de vida visuales. El layout del mapa se ajusto de grid a pack para corregir problemas de visibilidad del panel lateral. Estas adaptaciones demuestran la capacidad de

respuesta ante cambios en los requerimientos durante el ciclo de desarrollo.

Repositorio: <https://github.com/nazth703/defensa-y-asalto-base>